2 · Exp 1 + 2 — Pointers and pointer arithmetic

Manual Exp 1: "implement Pointers". Exp 2: "implement Pointer Arithmetic". Both have no user input, so they are the shortest programs in the lab.

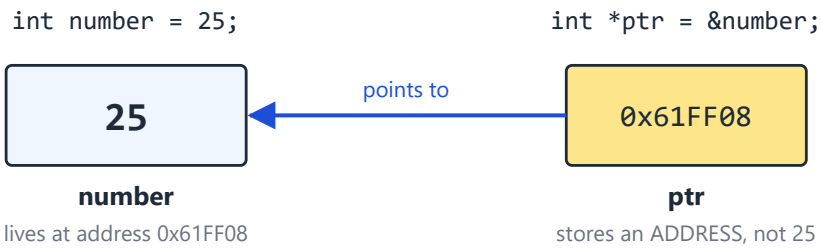
Address → Pointer variable → & and * → Pointer + 1 → Arrays are pointers → (later) linked lists

2.1 The idea

Every variable lives in memory at a numbered location: its **address**. A **pointer** is a variable whose value *is* an address.

GUESS FIRST

`int number = 25;` and `int *ptr = &number;`. If you print `ptr`, do you see 25? If you print `*ptr`?



Exp 1's variables. The address 0x61FF08 is the manual's example; yours will differ on every run.

Symbol	Read it as	In Exp 1
`int *ptr;`	"ptr is a pointer to an int"	declares the pointer
`&number`	"address of number"	`ptr = &number;`
`*ptr`	"value at the address in ptr"	prints 25

`*` MEANS TWO DIFFERENT THINGS

In a **declaration** (`int *ptr;`), `*` means "this is a pointer". In an **expression** (`cout << *ptr;`), `*` means "go to that address" (dereference).

2.2 Exp 1 program (manual's code was correct)

E01_POINTERS.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int number = 25;
    int *ptr;              // ptr can hold the address of an int

    ptr = &number;         // & = "address of"

    cout << "Value of number = " << number << endl;
    cout << "Address of number = " << &number << endl;
    cout << "Pointer stores = " << ptr << endl;
    cout << "Value using pointer = " << *ptr << endl;   // * = "value at"

    return 0;
}
```

SCREEN (REAL RUN)

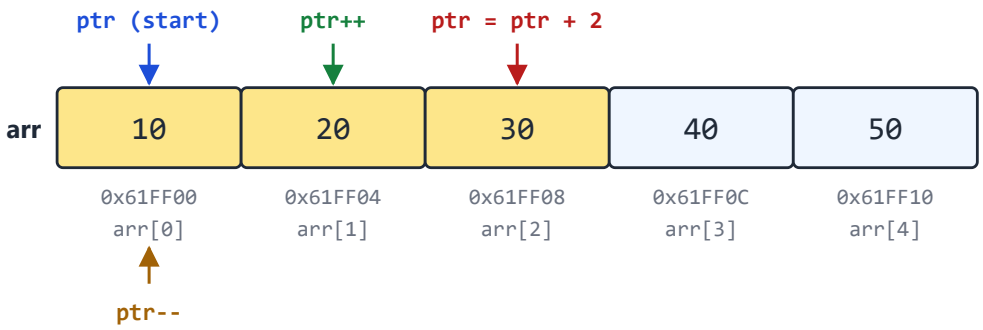
```
Value of number = 25
Address of number = 0xbb8cdff8c4
Pointer stores = 0xbb8cdff8c4
Value using pointer = 25
```

Lines 2 and 3 of the output always match each other; the actual number changes on every run and every PC. The manual says so too ("memory address varies").

2.3 Exp 2: pointer arithmetic

> **GUESS FIRST**
> `ptr` points at `arr[0]`, which is at address ...00. After `ptr++`, is the address ...01 or ...04? Why?

`ptr + 1` doesn't add 1 byte. It moves to the **next int**, which is `sizeof(int)` = 4 bytes further. That only works because an array's elements sit **side by side** in memory.



+1 on an int pointer = +4 bytes (sizeof(int)), i.e. the NEXT element

The manual's Figure 1, redrawn: each move is one whole element.

E02_POINTER_ARITHMETIC.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;                  // array name = address of arr[0]

    cout << "Initial Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr++;                           // moves 1 int forward (4 bytes)
    cout << "\nAfter Increment" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr--;                           // moves 1 int back
    cout << "\nAfter Decrement" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr = ptr + 2;                   // moves 2 ints forward (8 bytes)
    cout << "\nAfter Adding 2" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    return 0;
}
```

SCREEN (REAL RUN)

```
Initial Address : 0x45c67ffa90
Value : 10

After Increment
Address : 0x45c67ffa94
Value : 20

After Decrement
Address : 0x45c67ffa90
Value : 10

After Adding 2
Address : 0x45c67ffa98
Value : 30
```

The real run shows exactly the pattern the manual predicts: ...700 → ...704 → ...700 → ...708, values 10 → 20 → 10 → 30.

TWO FACTS THE VIVA LOVES

- `int *ptr = arr;` works because **an array's name is the address of its first element**: `arr == &arr[0]`.
- `*(ptr + i)` is the same as `arr[i]`. Square brackets are pointer arithmetic in disguise.

WRITE IN YOUR RECORD

Aim (Exp 1): To store the address of a variable in a pointer and access the variable through the pointer.

Algorithm: declare `int number = 25` → declare `int *ptr` → `ptr = &number` → print number, `&number`, `ptr`, `*ptr`.

Aim (Exp 2): To perform increment, decrement and addition on a pointer to an integer array.

Algorithm: `ptr = arr` → print → `ptr++` → print → `ptr--` → print → `ptr = ptr + 2` → print.

Complexity: time O(1), space O(1) for both.

Result (Exp 1): The pointer stored the address of `number` (same value as `&number`), and `*ptr` gave 25.

Result (Exp 2): Each increment moved the pointer by 4 bytes (one int): values 10, 20, 10, 30 at addresses base, +4, base, +8.

2.4 Viva questions

Question	Answer
What is a pointer?	A variable that stores the memory address of another variable.
What does `&` do? `*`?	`&x` gives x's address; `*p` gives the value stored at the address in p.
Why does `ptr++` add 4, not 1?	Pointer arithmetic moves in units of the pointed-to type; an int is 4 bytes here.
What would `ptr++` add for a `char *`? A `double *`?	1 byte; 8 bytes.
What is a NULL pointer?	A pointer that points nowhere (`ptr = NULL;`). Linked lists use it to mark the end.
What happens if you dereference an uninitialised pointer?	Undefined behaviour: it points to a random address, and the program may crash.
Where are pointers used?	Dynamic memory (`new`), linked lists, trees, graphs, passing by reference, arrays.

CHECK 2.1

`int a[4] = {5, 6, 7, 8}; int *p = a + 3;`. What is `*p`? What is `*(p - 2)`?

CHECK 2.2

Without looking, write the four lines of Exp 1 that declare `number`, declare `ptr`, assign it, and print the value through the pointer.

Answers

Check 2.1: `*p` = a[3] = **8**; `*(p - 2)` = a[1] = **6**.

Check 2.2: `int number = 25;` · `int *ptr;` · `ptr = &number;` · `cout << *ptr;`. If you wrote `ptr = number` (no `&`), the compiler rejects it: you can't store an int in a pointer.